

Vinheria IoT

Adega Climatizada da Vinheria Agnello

Equipe CodeSquad-FIAP

Roger Viana Gonçalves de Alencar

RM 97540 — Hardware e circuito

Kevin Benevides

Firmware Arduino e integração serial

Agosto de 2026

1 Objetivo e contextualização

O projeto apresenta uma solução de Internet das Coisas para monitorar a adega climatizada da Vinheria Agnello. A conservação de vinhos exige controle ambiental constante: variações de temperatura aceleram o envelhecimento, umidade inadequada prejudica as rolhas e a exposição luminosa pode provocar degradação fotoquímica da bebida.

A solução coleta temperatura, umidade e luminosidade, transmite as medições em JSON pela porta serial e permite que o Node-RED encaminhe os dados ao dashboard e a um broker MQTT.

2 Sensores e justificativas

2.1 Temperatura e umidade: DHT22

O DHT22 foi selecionado por fornecer temperatura e umidade em um único sensor digital. Para a conservação dos vinhos, a temperatura deve permanecer estável, idealmente entre 12°C e 18°C, enquanto a umidade deve ficar próxima de 70%. Um ambiente seco pode retrair as rolhas e permitir a entrada de oxigênio; um ambiente excessivamente úmido favorece fungos e danos aos rótulos.

2.2 Luminosidade: módulo LDR

O módulo LDR mede a incidência de luz na adega. Exposição luminosa excessiva, especialmente à radiação ultravioleta, pode desencadear reações nos compostos de enxofre do vinho e produzir o defeito conhecido como *light strike*. O monitoramento permite detectar iluminação esquecida ou entrada indevida de luz externa.

3 Arquitetura de hardware

O circuito é simulado no Wokwi com um Arduino Uno e possui a seguinte pinagem:

- DHT22: alimentação em 5 V, terra comum e dados no pino digital D2;
- módulo LDR: alimentação em 5 V, terra comum e saída analógica no A0.

O arquivo executável do circuito está versionado em `Arduino/VinheriaSensores/diagram.json`. A listagem completa aparece na Seção 7.

4 Firmware e formato dos dados

O firmware inicializa o DHT22, configura o LDR e realiza uma amostragem a cada 4000 ms. O intervalo é controlado com `millis()`, sem bloquear o microcontrolador com `delay()`.

Apesar de o circuito possuir dois sensores físicos, o DHT22 produz duas medições. Assim, cada mensagem contém três campos semânticos:

```
1 {"temperatura":14.2,"umidade":70.0,"luminosidade":512}
```

Listing 1: Mensagem serial produzida pelo firmware

Temperatura é expressa em graus Celsius, umidade em percentual e luminosidade como leitura bruta do conversor analógico do Arduino Uno, entre 0 e 1023. Se o DHT22 falhar, temperatura e umidade são publicadas como `null`; desse modo o documento continua sendo JSON válido e a leitura independente do LDR não é perdida.

5 Integração com Node-RED e MQTT

O Arduino Uno utilizado não possui interface de rede. Portanto, a publicação MQTT é responsabilidade do Node-RED, conforme o fluxo:

Arduino Uno → Serial → Node-RED → HiveMQ → Dashboard

A serial opera em 9600 baud, 8 bits de dados, sem paridade e um bit de parada. Cada mensagem termina com `\n`. No Node-RED, um nó serial separa as linhas, um nó JSON converte a mensagem e os valores podem alimentar gráficos ou ser publicados em um tópico MQTT.

Essa separação mantém o firmware responsável apenas pela aquisição e pelo protocolo serial, enquanto conectividade, armazenamento e visualização ficam na camada de integração.

6 Reprodução e validação

O diretório `Arduino/VinheriaSensores` reúne o sketch, o circuito e a lista de bibliotecas. Para reproduzir a simulação:

1. abrir um projeto Arduino Uno no Wokwi;
2. copiar `VinheriaSensores.ino`, `diagram.json` e `libraries.txt`;
3. iniciar a simulação e abrir o monitor serial;
4. alterar temperatura, umidade e luminosidade no simulador;
5. confirmar uma linha JSON válida a cada quatro segundos.

Para a compilação local, utiliza-se o Arduino CLI com o core AVR e a biblioteca DHT da Adafruit.

7 Códigos-fonte

Os arquivos abaixo são incluídos diretamente no relatório para evitar que a documentação fique diferente da versão executável.

7.1 Esquema de hardware

```
1 {
2   "version": 1,
3   "author": "CodeSquad-FIAP",
4   "editor": "wokwi",
5   "parts": [
6     {
7       "type": "wokwi-arduino-uno",
8       "id": "uno",
9       "top": 0,
10      "left": 0,
11      "attrs": {}
12    },
13    {
14      "type": "wokwi-dht22",
15      "id": "dht1",
16      "top": -150,
17      "left": -200,
18      "attrs": {
19        "temperature": "14",
20        "humidity": "70"
21      }
22    },
23    {
24      "type": "wokwi-photoresistor-sensor",
25      "id": "ldr1",
26      "top": -150,
27      "left": 100,
28      "attrs": {
29        "lux": "10"
30      }
31    }
32  ],
33  "connections": [
34    ["dht1:VCC", "uno:5V", "red", ["v0"]],
35    ["dht1:GND", "uno:GND.1", "black", ["v0"]],
36    ["dht1:SDA", "uno:2", "green", ["v0"]],
37    ["ldr1:VCC", "uno:5V", "red", ["v0"]],
38    ["ldr1:GND", "uno:GND.2", "black", ["v0"]],
39    ["ldr1:A0", "uno:A0", "yellow", ["v0"]]
40  ]
41 }
```

Listing 2: Circuito Wokwi em diagram.json

7.2 Firmware Arduino

```
1 #include <DHT.h>
2
3 /**
4  * Vinheria Agnello - Monitoramento da adega climatizada
```

```

5  *
6  * Sensores definidos pelo projeto de hardware:
7  *   - DHT22 no pino digital D2: temperatura e umidade;
8  *   - Modulo LDR no pino analogico A0: luminosidade bruta.
9  *
10 * A cada quatro segundos, o firmware publica uma unica linha JSON pela porta
11 * serial. O Node-RED pode separar as mensagens por quebra de linha, converter
12 * o JSON e encaminhar os dados ao dashboard ou ao broker MQTT.
13 */
14
15 // Pinagem definida no circuito do Wokwi.
16 const uint8_t DHT_PIN = 2;
17 const uint8_t LDR_PIN = A0;
18 const uint8_t DHT_TYPE = DHT22;
19
20 // Configuracoes compartilhadas com o monitor serial e com o Node-RED.
21 const unsigned long SERIAL_BAUD_RATE = 9600UL;
22 const unsigned long INTERVALO_LEITURA_MS = 4000UL;
23
24 // Objeto responsavel pela comunicacao com o sensor digital DHT22.
25 DHT dht(DHT_PIN, DHT_TYPE);
26
27 // millis() retorna unsigned long. Usar o mesmo tipo permite que a verificacao
28 // de intervalo continue correta mesmo quando o contador interno transbordar.
29 unsigned long instanteUltimaLeitura = 0UL;
30
31 /**
32  * Inicializa a comunicacao serial e os sensores.
33  *
34  * Nenhuma mensagem de inicializacao e enviada pela serial, pois cada linha da
35  * saida deve ser um JSON valido para ser consumido automaticamente.
36  */
37 void setup() {
38   Serial.begin(SERIAL_BAUD_RATE);
39   dht.begin();
40   pinMode(LDR_PIN, INPUT);
41 }
42
43 /**
44  * Imprime um numero do DHT22 ou null quando a leitura for invalida.
45  *
46  * JSON nao reconhece NaN como numero. Publicar null mantem a mensagem valida e
47  * permite que o fluxo no Node-RED trate uma falha sem descartar a luminosidade.
48  */
49 void imprimirMedicaoDht(const float valor, const bool leituraValida) {
50   if (leituraValida) {
51     // Uma casa decimal e suficiente para a precisao nominal do DHT22.
52     Serial.print(valor, 1);
53     return;
54   }
55
56   Serial.print(F("null"));
57 }
58
59 /**
60  * Envia as medicoes em uma unica linha JSON terminada por \n.
61  *
62  * Exemplo de sucesso:

```

```

63 * {"temperatura":14.2,"umidade":70.0,"luminosidade":512}
64 *
65 * Em caso de falha do DHT22, temperatura e umidade recebem null, mas a leitura
66 * independente do LDR continua disponivel. O campo erro facilita diagnostico.
67 */
68 void enviarLeiturasComoJson(
69     const float temperatura,
70     const float umidade,
71     const int luminosidade,
72     const bool leituraDhtValida) {
73     Serial.print(F("{\"temperatura\":"));
74     imprimirMedicaoDht(temperatura, leituraDhtValida);
75     Serial.print(F(",\"umidade\":"));
76     imprimirMedicaoDht(umidade, leituraDhtValida);
77     Serial.print(F(",\"luminosidade\":"));
78     Serial.print(luminosidade);
79
80     if (!leituraDhtValida) {
81         Serial.print(F(",\"erro\": \"Falha de leitura do DHT22\""));
82     }
83
84     Serial.println(F("}"));
85 }
86
87 /**
88 * Executa a amostragem sem bloquear o microcontrolador com delay().
89 *
90 * O DHT22 e o LDR sao lidos somente depois que os 4000 ms solicitados passam.
91 * Durante o restante do tempo, loop() retorna imediatamente e deixa a placa
92 * livre para futuras funcionalidades.
93 */
94 void loop() {
95     const unsigned long instanteAtual = millis();
96
97     if (instanteAtual - instanteUltimaLeitura < INTERVALO_LEITURA_MS) {
98         return;
99     }
100
101     instanteUltimaLeitura = instanteAtual;
102
103     const float temperatura = dht.readTemperature();
104     const float umidade = dht.readHumidity();
105     const int luminosidade = analogRead(LDR_PIN);
106     const bool leituraDhtValida = !isnan(temperatura) && !isnan(umidade);
107
108     enviarLeiturasComoJson(
109         temperatura,
110         umidade,
111         luminosidade,
112         leituraDhtValida);
113 }

```

Listing 3: Firmware VinheriaSensores.ino